

Lambda & API Gateway

SnapStart · Concurrency · DLQ · HTTP API · VPC Link · WebSocket

AWS Tutorial · ShopFlow

09-00

Chapter Overview — Lambda & API Gateway at ShopFlow

ShopFlow uses AWS Lambda for event-driven processing (order stream, image resize, search indexing) and API Gateway as the managed HTTP front-door for the public REST API. ECS Fargate handles synchronous, stateful microservices; Lambda handles async, stateless event functions.

ShopFlow Function	Trigger	Runtime	Memory / Timeout	Invocations/mo
OrderStreamProcessor	DynamoDB Streams	Java 21 (SnapStart)	512MB / 5min	1M (every order event)
ImageResizer	S3 ObjectCreated	Java 21 (SnapStart)	1024MB / 30s	200K (product images)
SearchIndexer	SQS (from DDB Streams)	Java 21	256MB / 30s	500K (product changes)
PriceCalculator	API Gateway HTTP API	Java 21 (SnapStart)	256MB / 15s	5M (on product page load)
EmailDispatcher	SQS Standard	Java 21	128MB / 30s	200K (order confirmations)
DailyReport	EventBridge Scheduler	Java 21	512MB / 15min	30 (once per day)
Auroralnit (custom resource)	CloudFormation deploy	Python 3.12	128MB / 15min	1 (on stack create/update)

API Gateway Component	ShopFlow Use	Endpoint Count	Auth
HTTP API (v2)	PriceCalculator, public product search API	12 routes (GET /products, POST /cart/*, ...)	Cognito JWT authorizer
REST API (v1)	Admin dashboard API (requires request validation, WAF)	3 routes	IAM + Cognito
WebSocket API	Real-time order status updates to browser	2 routes (\$connect, order-status)	Cognito JWT

■ HTTP API (v2) is 70% cheaper than REST API (v1) and has lower latency. Use HTTP API for: JWT auth, Lambda proxy, CORS, route-level throttling. Use REST API only when you need: request validation, request/response transformation, usage plans, AWS WAF direct integration, or custom gateway responses. ShopFlow uses HTTP API for all public routes and REST API for admin routes requiring WAF.

09-01

Lambda Execution Model — Cold Start, Warm, SnapStart

Lambda lifecycle: INIT (cold start) → INVOKE (execute handler) → potential SHUTDOWN. Cold start occurs when no warm execution environment exists. Java 21 cold starts take 2-8s without optimization. SnapStart eliminates cold start by pre-initializing and snapshotting the JVM.

Lambda Execution Model — Cold vs Warm

Cold Start (new execution environment)

1. Download code package or container image (1-3s)
2. Start JVM, load classes, Spring context init (2-5s for Java)
3. Execute init code (static blocks, @PostConstruct) (0.5-2s)
4. Run handler (actual function logic)

Total cold start: 3-10s — unacceptable for API Gateway <1s SLA

Warm Invocation (reusing existing execution environment)

- Handler runs immediately — no class loading or JVM init
- Latency: 10-200ms (pure function logic)

SnapStart (ShopFlow — all Java Lambdas except SQS triggers)

- Snapshot JVM state after init phase once
- Resume from snapshot: sub-1s cold start (50-200ms)

SnapStart CDK Config

```
const priceCalc = new lambda.Function(this, "PriceCalculator", {
  functionName: "shopflow-price-calculator",
  runtime: lambda.Runtime.JAVA_21,
  handler: "com.shopflow.PriceHandler::handleRequest",
  code: lambda.Code.fromEcr(ecrRepo, {tag: "price-calc-v1.2.3"}),
  memorySize: 256,
  timeout: Duration.seconds(15),
  snapStart: lambda.SnapStartConf.ON_PUBLISHED_VERSIONS,
  environment: {
    REDIS_ENDPOINT: redis.attrPrimaryEndPointAddress,
    DB_SECRET_ARN: auroraSecret.secretArn,
  },
});
// SnapStart requires publishing a VERSION — use alias
const priceCalcAlias = new lambda.Alias(this, "PriceCalcProd", {
  aliasName: "prod",
  version: priceCalc.currentVersion,
});
```

Cold Start Optimization	Technique	Latency Improvement	Tradeoff
SnapStart (ShopFlow)	Pre-snapshot JVM state after init	3-8s → 50-200ms (95%+ reduction)	Only on published versions; init code runs once
Provisioned Concurrency	Keep N warm environments ready	0ms cold start (warm pool)	Cost: pay for idle time; complex scaling
Minimize dependencies	Remove unused Spring modules	Reduces class loading 20-40%	Requires code refactoring
GraalVM native image	Compile to native binary (no JVM)	<100ms cold start	Complex build; no dynamic class loading

■ SnapStart snapshots are encrypted with AWS KMS. The snapshot includes the JVM heap state after the INIT phase. Uniqueness hooks (Lambda CRaC interfaces) allow code to run on restore — use to re-establish external connections (Redis, DB) since network state is not preserved in the snapshot. Add implements CracHook to your Lambda handler and re-connect in afterRestore().

Java Lambda handlers implement `RequestHandler<I,O>` (buffered) or `RequestStreamHandler` (streaming). ShopFlow uses `RequestHandler` for structured API Gateway events and `RequestStreamHandler` with response streaming for the `DailyReport` function to avoid 6MB response limit.

API Gateway Proxy Integration — Java Handler

```
@Component
public class PriceHandler
implements RequestHandler<APIGatewayProxyRequestEvent, APIGatewayProxyResponseEvent> {
    private final PriceService priceService;
    private final ObjectMapper json = new ObjectMapper();
    // SnapStart: initialize Spring context in static init (before snapshot)
    private static final ApplicationContext ctx;
    static {
        ctx = new AnnotationConfigApplicationContext(AppConfig.class);
    }
    public PriceHandler() {
        this.priceService = ctx.getBean(PriceService.class);
    }
    @Override
    public APIGatewayProxyResponseEvent handleRequest(
        APIGatewayProxyRequestEvent event, Context context) {
        try {
            String productId = event.getPathParameters().get("productId");
            String userId = event.getRequestContext().getAuthorizer()
                .getJwt().getClaims().get("sub");
            PriceDto price = priceService.getPersonalizedPrice(productId, userId);
            return new APIGatewayProxyResponseEvent()
                .withStatusCode(200)
                .withHeaders(Map.of("Content-Type", "application/json",
                    "Cache-Control", "max-age=60"))
                .withBody(json.writeValueAsString(price));
        } catch (ProductNotFoundException e) {
            return new APIGatewayProxyResponseEvent()
                .withStatusCode(404)
                .withBody("{\"error\":\"Product not found\"}");
        }
    }
}
```

■ Initialize Spring `ApplicationContext` in a static `{}` block before SnapStart snapshots the JVM. Post-snapshot, the context is already built — your handler constructor just fetches beans from the existing context. Without this pattern, Spring starts on every cold start (even with SnapStart, the first invocation after restore would re-initialize Spring if it was initialized lazily).

Lambda concurrency: at most 1 invocation per execution environment simultaneously. Reserved concurrency guarantees N environments for one function (and limits it to N — provides isolation). Provisioned concurrency pre-warms N environments to eliminate cold starts for latency-sensitive functions.

Concurrency Type	What It Does	Cost	ShopFlow Use
Unreserved (default)	Shares account's 1,000 limit across all functions	Pay only per invocation	SQS, Streams, async functions
Reserved concurrency	Guarantees N for this function; caps at N	Pay only per invocation (reserved capacity fee)	Price Calculator: reserve 200 (prevents stale data)
Provisioned concurrency	Keeps N warm — zero cold start, instant response	Pay per invocation × N environments + per-minute fee	API Gateway: 10 provisioned for p99 < 1s
Account burst limit	1,000 → +500/min until 3,000 (us-east-1)	N/A	Burst plan: pre-warm 1 week before Black Friday

Concurrency CDK Config

```
// Reserve 200 concurrent invocations for PriceCalculator
// Prevents high-traffic function from starving other functions
priceCalcAlias.addAutoScaling({
  minCapacity: 5,
  maxCapacity: 200,
}).scaleOnUtilization({
  utilizationTarget: 0.7, // scale up when 70% of reserved is consumed
});
// Provisioned Concurrency: 20 always-warm instances for p99 SLA
const pcProvision = priceCalcAlias.addAutoScaling({
  minCapacity: 20, // always-warm floor
  maxCapacity: 200, // scale beyond provisioned with SnapStart for remainder
});
pcProvision.scaleOnSchedule("BlackFridayMorning",{
  schedule: appscaling.Schedule.expression("cron(0 7 29 11 ? *)"),
  minCapacity: 100, // pre-warm 100 on Black Friday 7am
  maxCapacity: 500,
});
```

Scenario	Without Provisioned Concurrency	With Provisioned Concurrency (20)
First request after quiet period	Cold start: 200-500ms (SnapStart)	Warm: <50ms
Black Friday 9am burst (1,000 requests)	20 cold starts first second	0 cold starts up to 20 concurrent
20→500 concurrent ramp	SnapStart cold starts: ~200ms each	First 20 warm; 21-500 use SnapStart
Cost (20 provisioned × 730 hr/mo)	\$0	20 × \$0.000004646 × 3600 × 730 = \$244/mo

■ Provisioned concurrency for Java Lambda with Spring context can be expensive at high counts. 100 provisioned instances × 512MB = 51GB of memory reserved. Cost: 100 × \$0.000004646/GB-s × 512 × 3600 × 730 = ~\$630/mo. Evaluate whether SnapStart (50-200ms cold) is sufficient before committing to provisioned concurrency. Most ShopFlow use-cases tolerate SnapStart latency.

09-04

Lambda — Aurora & Redis Access (Connection Reuse)

Lambda execution environments are reused across invocations — database connections initialized outside the handler persist for the lifetime of the warm environment. Reusing connections avoids Aurora connection limit exhaustion. RDS Proxy is mandatory for Lambda-to-Aurora connectivity at scale.

Connection Reuse Pattern — Outside Handler

```
public class SearchIndexerHandler
implements RequestHandler<SQSEvent, Void> {
  // OUTSIDE handler – initialized ONCE per execution environment
  // Reused across 1000s of invocations until environment is recycled
  private static final DataSource dataSource;
  private static final RedisTemplate<String, Object> redis;
  static {
    // RDS Proxy endpoint – connection multiplexing
    HikariConfig hc = new HikariConfig();
    hc.setJdbcUrl(System.getenv("RDS_PROXY_ENDPOINT"));
    hc.setMaximumPoolSize(1); // Lambda: 1 concurrent connection per env
    hc.setConnectionTimeout(10_000);
    hc.setIdleTimeout(60_000); // Return idle connections quickly
    // IAM Auth: token generated on first connect, cached 15 min
    hc.addDataSourceProperty("sslMode", "require");
    dataSource = new HikariDataSource(hc);
    // Redis: Lettuce connection (non-blocking, single connection)
    LettuceConnectionFactory lcf = new LettuceConnectionFactory(
      System.getenv("REDIS_ENDPOINT"), 6379);
```

```

lcf.afterPropertiesSet();
redis = new RedisTemplate<>();
redis.setConnectionFactory(lcf);
redis.afterPropertiesSet();
}

@Override
public Void handleRequest(SQSEvent event, Context context) {
// Reuses static dataSource and redis – no re-initialization
event.getRecords().forEach(r -> indexProduct(r.getBody()));
return null;
}
}

```

Database Access Pattern	Aurora Connections Used	Risk	ShopFlow Solution
Direct JDBC, connection per invocation	1 per invocation (immediate expiration)	High concurrency = 1,000 connections – Aurora can't handle	Aurora always use RDS Proxy
HikariCP pool, max=5, no proxy	Up to 5 per execution env × 1,000 r6g.large = 5,000	max_connections = 170 (r6g.large)	Use RDS Proxy to multiplex
RDS Proxy + HikariCP max=1 (ShopFlow)	1 Proxy multiplexes: 1,000 Lambdas	Safe – RDS Proxy holds 80 persistent connections to Aurora	Connections to Aurora efficient

■ RDS Proxy connection multiplexing: 1,000 Lambda execution environments each holding one Proxy connection = 1,000 connections to the Proxy. Proxy maintains only 30 connections to Aurora (configurable). The Proxy queues requests and pin connections only for transactions with session state. Effective: Lambda concurrency can scale to thousands without exhausting Aurora's connection limit.

09-05

API Gateway HTTP API — Routes, Auth & CORS

ShopFlow public API uses API Gateway HTTP API (v2): Cognito JWT authorizer validates Bearer tokens, route-level throttling protects Lambda from burst traffic, and CORS is configured per-route for browser-based fetch calls from the React frontend.

HTTP API CDK

```

const api = new apigwv2.HttpApi(this, "ShopFlowApi", {
  apiName: "shopflow-api",
  corsPreflight: {
    allowOrigins: ["https://shopflow.com", "https://www.shopflow.com"],
    allowMethods: [apigwv2.CorsHttpMethod.GET, apigwv2.CorsHttpMethod.POST,
      apigwv2.CorsHttpMethod.PUT, apigwv2.CorsHttpMethod.DELETE],
    allowHeaders: ["Content-Type", "Authorization"],
    maxAge: Duration.hours(1),
  },
  defaultThrottle: {
    rateLimit: 10_000, // steady-state: 10K req/s across all routes
    burstLimit: 15_000, // burst: 15K req/s for 1-2 seconds
  },
});

// Cognito JWT authorizer
const jwtAuth = new apigwv2.HttpJwtAuthorizer("CognitoJwt",
  `https://cognito-idp.us-east-1.amazonaws.com/${userPool.userPoolId}`, {
  jwtAudience: [userPoolClient.userPoolClientId],
});

// Routes
api.addRoutes({
  path: "/products/{productId}/price",
  methods: [apigwv2.HttpMethod.GET],
  integration: new HttpLambdaIntegration("Price", priceCalcAlias),
  authorizer: jwtAuth,
});

```

Route	Method	Lambda Function	Auth	Throttle Override
GET /products	GET	No Lambda — ECS ALB (API GW not used)	None	—
GET /products/{id}/price	GET	PriceCalculator (SnapStart)	Cognito JWT	5,000 RPS
POST /cart/{userId}/items	POST	ECS ALB (via VPC Link)	Cognito JWT	1,000 RPS
GET /search/autocomplete	GET	SearchAutocomplete Lambda	None (public)	500 RPS — expensive Redis query
POST /orders	POST	ECS ALB (complex order logic)	Cognito JWT	2,000 RPS
DELETE /cart/{userId}/items/{id}	DELETE	ECS ALB (via VPC Link)	Cognito JWT	500 RPS

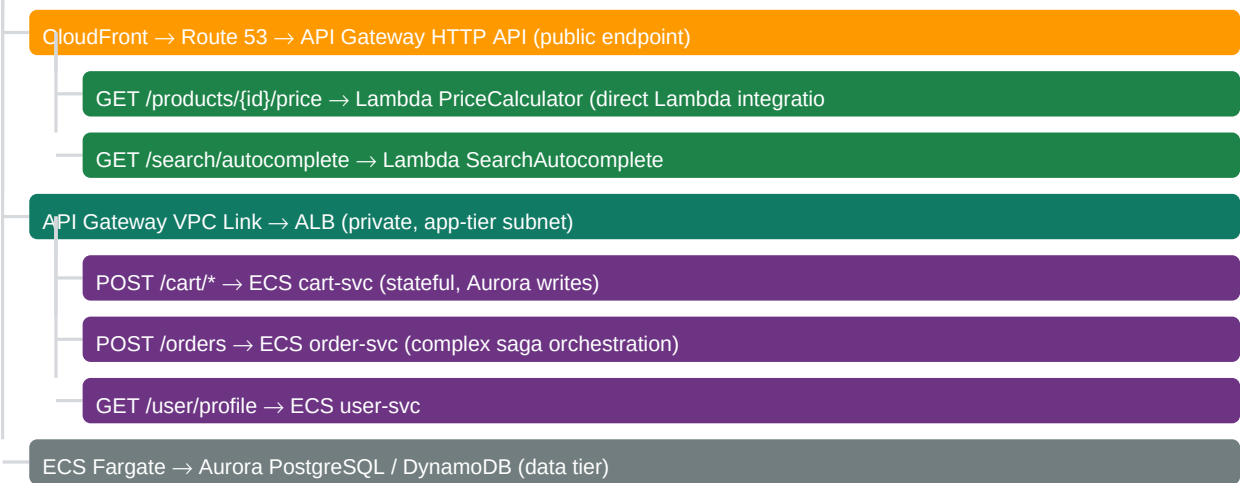
■ API Gateway HTTP API charges \$1.00 per million API calls + \$0.0035 per GB data transfer. REST API charges \$3.50 per million. For ShopFlow at 5M daily API calls: HTTP API cost = \$5/day vs REST API = \$17.50/day. Save \$375/month by using HTTP API for all routes that do not require REST API-specific features (request validation, usage plans, WAF integration).

09-06

API Gateway VPC Link — Routing to ECS Fargate

API Gateway VPC Link connects HTTP API directly to the ECS Application Load Balancer in the private VPC. Cart and order routes use VPC Link to route to ECS Fargate services without exposing the ALB publicly. Lambda handles compute-light, stateless functions; ECS handles stateful service logic.

ShopFlow Request Routing Architecture



VPC Link CDK

```
// VPC Link: private connection from API GW to ALB in app subnet
const vpcLink = new apigwv2.VpcLink(this, "VpcLink", {
  vpc,
  subnets: vpc.selectSubnets({subnetType: SubnetType.PRIVATE_WITH_EGRESS}),
  securityGroups: [sgVpcLink],
  vpcLinkName: "shopflow-private-link",
});

// Cart routes → ECS via VPC Link
api.addRoutes({
  path: "/cart/{userId}/items",
  methods: [apigwv2.HttpMethod.POST],
  integration: new HttpAlbIntegration("CartAlb", cartAlbListener, {
    vpcLink,
    parameterMapping: new apigwv2.ParameterMapping()
      .appendHeader("x-shopflow-user-id",
        apigwv2.MappingValue.contextVariable("authorizer.claims.sub")),
  }),
});
```

```
    }},
    authorizer: jwtAuth,
  });
```

Integration Type	Target	Latency Added	Use When
Lambda (direct)	Lambda function or alias	~1ms API GW overhead	Stateless compute; SnapStart available; short-lived
HTTP (public URL)	Any public HTTP endpoint	~2ms + network RTT	External SaaS APIs; legacy HTTP services
ALB (via VPC Link)	Private ALB in VPC	~2ms + VPC Link RTT	ECS/EKS services in private VPC; complex business
Service Discovery (via VPC Link)	Private Cloud Map service	~2ms + VPC Link RTT	ECS services registered in Cloud Map

■ VPC Link charges \$0.01 per hour (\$7.30/month) per VPC Link, plus \$0.01 per GB data processed. For ShopFlow cart API (100K POST /cart requests/day × avg 1KB body): \$0.01/GB × 0.1GB/day × 30 = \$0.03/month data cost. The \$7.30/month VPC Link charge is negligible for the security benefit of keeping ALB private.

09-07

Lambda Dead Letter Queue & Retry Behavior

Lambda retries differ by trigger type. Synchronous (API Gateway): no retry — caller receives the error. Asynchronous (S3, EventBridge): retries up to 2× with exponential backoff, then routes to DLQ. Event source mapping (SQS, DynamoDB Streams, Kinesis): retry until success or record expires — DLQ/bisectOnError critical.

Trigger Type	Retry Behavior	DLQ Applicable	ShopFlow Config
API Gateway (sync)	No retry — error returned to caller in <29s	No — errors are API response errors in response body; 4xx/5xx	
S3 Event (async)	2 retries with exponential backoff	Yes — S3 → Lambda DLQ sized fail DLQ = shopflow-image-dlq	
EventBridge (async)	Configurable 0-185 retry + max event age	Yes	DailyReport: retry=0 (cron, run next day)
SQS trigger	Retry until maxReceiveCount, then SQS DLQ	SQS DLQ (not Lambda DLQ)	SqsDLQIndexer: maxReceiveCount=3, DLQ retry=0
DynamoDB Streams	Retry until record expires (24hr) or maxRetry	Lambda DLQ (after maxRetry)	StreamProcessor: bisectOnError=true, retry=0
Kinesis	Retry until record expires (24hr) or maxRetry	Lambda DLQ	EventIngester: bisectOnError=true, parallelization=1

ImageResizer DLQ CDK — Async Lambda with DLQ

```
const imageDlq = new sqs.Queue(this,"ImageResizerDlq",{
  queueName: "shopflow-image-resizer-dlq",
  retentionPeriod: Duration.days(14),
  encryption: sqs.QueueEncryption.KMS_MANAGED,
});

const imageResizer = new lambda.Function(this,"ImageResizer",{
  functionName: "shopflow-image-resizer",
  runtime: lambda.Runtime.JAVA_21,
  handler: "com.shopflow.ImageResizerHandler::handleRequest",
  code: lambda.Code.fromEcr(ecrRepo,{tag:"image-resizer-v1"}),
  memorySize: 1024,
  timeout: Duration.seconds(30),
  snapStart: lambda.SnapStartConf.ON_PUBLISHED_VERSIONS,
  deadLetterQueue: imageDlq,
  retryAttempts: 2,
  onSuccess: new destinations.SqsDestination(successQueue),
});

imageResizer.addEventSource(new S3EventSource(productImageBucket,{
  events: [s3.EventType.OBJECT_CREATED],
  filters: [{ prefix:"raw/", suffix:".jpg" }],
}));
```

■ Always configure a DLQ for all async Lambda invocations (S3, EventBridge, SNS). Without DLQ: after 2 retries, Lambda silently drops the event and logs the error to CloudWatch. The failure is invisible unless you specifically query CloudWatch for ThrottleErrors or DeadLetterErrors. With DLQ: failed events accumulate in SQS for reprocessing, alerting, and

root-cause analysis.

09-08

API Gateway Throttling, Usage Plans & WAF

API Gateway throttling prevents Lambda from being overwhelmed by burst traffic. Route-level throttle limits protect expensive endpoints. AWS WAF integration on REST API blocks common web attacks (SQL injection, XSS, rate limiting by IP). HTTP API requires CloudFront + WAF for layer-7 protection.

Throttling Level	Where Applied	ShopFlow Config	Effect of Limit
Account-level	All API GW in the region	10,000 RPS (AWS default) — cannot lower	Requests beyond limit receive 429
Stage-level	All routes in one stage (prod/dev)	5,000 RPS default in prod stage	Stage-wide rate limit
Route-level	Specific route override	GET /search/autocomplete: 500 RPS	Prevent expensive route from starting
Usage plan (REST API only)	Per API Key (partner APIs)	Admin API: 100 RPS per partner key	Per-customer rate limiting; billing
Lambda reserved concurrency	Lambda, not API GW layer	PriceCalculator: max 200 concurrent	Limits downstream Lambda execution

WAF WebACL for REST API Admin Routes

```
const wafAcl = new wafv2.CfnWebACL(this, "AdminWaf", {
  name: "shopflow-admin-api-waf",
  scope: "REGIONAL", // REST API WAF: REGIONAL (NOT us-east-1 global)
  defaultAction: { allow: {} },
  rules: [
    // 1. AWS Managed Rules – common web exploits
    { name: "AWSManagedRulesCommonRuleSet",
      overrideAction: { none: {} },
      visibilityConfig: {
        cloudWatchMetricsEnabled: true,
        metricName: "AWSManagedRulesCommonRuleSetMetric",
        sampledRequestsEnabled: true,
      },
      statement: {
        managedRuleGroupStatement: {
          vendorName: "AWS", name: "AWSManagedRulesCommonRuleSet"
        },
      },
      priority: 1 },
    // 2. Rate limiting: 100 requests per 5min per IP
    { name: "AdminRateLimitByIp",
      action: { block: {} },
      statement: { rateBasedStatement: { limit: 100,
        aggregateKeyType: "IP" } },
      priority: 2,
      visibilityConfig: { cloudWatchMetricsEnabled: true,
        metricName: "AdminRateLimit", sampledRequestsEnabled: true },
    },
    { name: "ShopFlowAdminWaf",
      action: { block: {} },
      statement: { ruleSetReferenceStatement: { arn: "arn:aws:wafv2:us-east-1:123456789012:webacl/shopflow-admin-api-waf" } },
      priority: 3,
      visibilityConfig: { cloudWatchMetricsEnabled: true,
        metricName: "ShopFlowAdminWaf", sampledRequestsEnabled: true },
    },
  ],
});
```

■ API Gateway REST API WAF must be REGIONAL scope. CloudFront WAF must be GLOBAL scope and deployed in us-east-1 region only. ShopFlow uses CloudFront WAF (global, us-east-1) for the main HTTP API and a REGIONAL WAF for the REST API admin routes. Attempting to attach a GLOBAL WAF to a REST API in ap-southeast-1 will fail — scope must match deployment type.

09-09

Lambda Layers — Shared Dependencies

Lambda Layers allow sharing code and dependencies across functions without bundling them in each deployment package. ShopFlow uses a shared-libraries layer for Jackson, AWS SDK v2, and monitoring utilities — reducing each

function deployment package from 45MB to 8MB.

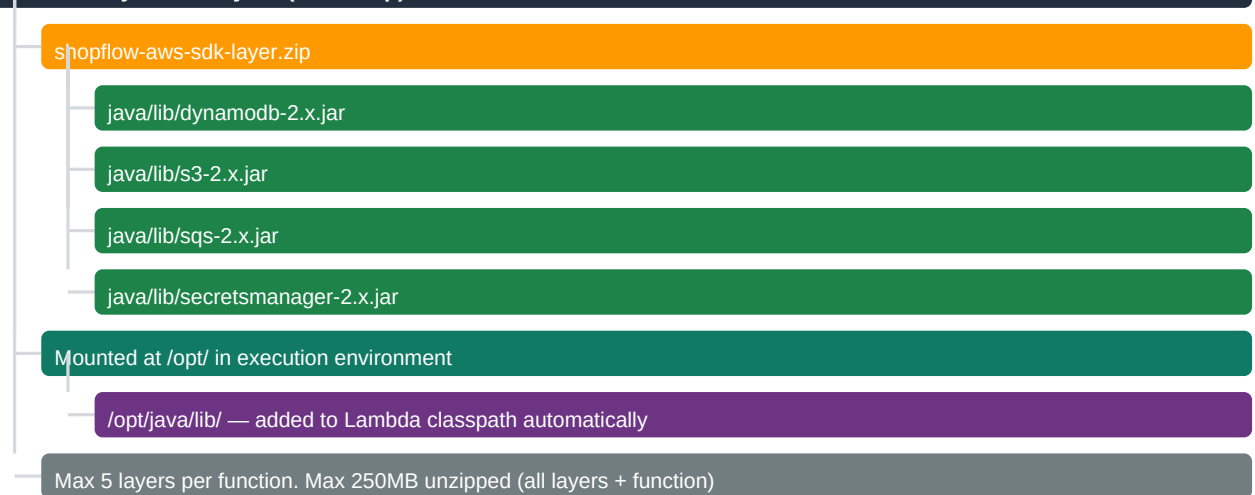
Layer Name	Contents	Functions Using It	Size Reduction
shopflow-aws-sdk-layer	AWS SDK v2 (DynamoDB, S3, SQS, Secrets Manager)	All 7 functions	35MB removed from each package
shopflow-jackson-layer	Jackson 2.15, ObjectMapper, JSON modules	All 7 functions	8MB removed from each package
shopflow-observability-layer	AWS X-Ray SDK, CloudWatch EMF, Powertools for Lambda	All 7 functions	5MB removed from each package
shopflow-db-layer	HikariCP, Aurora JDBC driver, Flyway client	SearchIndexer, OrderStream	12MB removed

Lambda Layer CDK

```
// Create shared AWS SDK layer
const awsSdkLayer = new lambda.LayerVersion(this, "AwsSdkLayer", {
  layerVersionName: "shopflow-aws-sdk-layer",
  code: lambda.Code.fromAsset("layers/aws-sdk"),
  compatibleRuntimes: [lambda.Runtime.JAVA_21],
  compatibleArchitectures: [lambda.Architecture.ARM_64],
  description: "AWS SDK v2 shared across all ShopFlow Lambdas",
});

// Attach to function
priceCalcFn.addLayers(awsSdkLayer, jacksonLayer, observabilityLayer);
// Layers are read-only at /opt/java/lib/ in execution environment
// Maven: exclude AWS SDK from function pom.xml (provided scope)
// <dependency>
// <groupId>software.amazon.awssdk</groupId>
// <artifactId>dynamodb</artifactId>
// <scope>provided</scope> <-- not bundled in JAR
// </dependency>
```

Lambda Layer File Layout (inside zip)



■ Layers add cold start overhead because the JVM must load additional JARs from /opt/. For Java functions using SnapStart, layers are included in the snapshot — so layer JARs are loaded ONCE during snapshot creation, not on every cold start. For Java Lambdas without SnapStart, minimize layer count and size to reduce JVM class loading time on cold start.

Lambda pricing: \$0.0000000083 per GB-millisecond. Increasing memory reduces duration (faster execution) — sometimes lower-total cost. AWS Lambda Power Tuning tool tests 6+ memory configurations and recommends the optimal setting for cost, speed, or balanced objective.

Memory (MB)	Duration (ms) for SearchIndexer	Cost per invocation	Monthly (500K invocations)
128	850	$128/1024 \times 0.850 \times \$0.0000000083 = \$0.000000089$	\$0.00000089
256	480	$256/1024 \times 0.480 \times \$0.0000000083 = \$0.000000100$	\$0.00000100
512 (balanced)	260	$512/1024 \times 0.260 \times \$0.0000000083 = \$0.000000108$	\$0.00000108
1024	140	$1024/1024 \times 0.140 \times \$0.0000000083 = \$0.000000116$	\$0.00000116
1792 (1 vCPU bound)	80	$1792/1024 \times 0.080 \times \$0.0000000083 = \$0.000000116$	\$0.00000116

ARM64/Graviton2 for Lambda — 20% Cost Reduction

```
// CDK: use ARM_64 architecture (Graviton2) — 20% cheaper than x86_64
const imageResizer = new lambda.Function(this, "ImageResizer", {
  architecture: lambda.Architecture.ARM_64, // Graviton2
  runtime: lambda.Runtime.JAVA_21, // Java 21 supports ARM_64
  memorySize: 512,
  // ... other config
});

// Graviton2 pricing: $0.0000000067 per GB-ms (vs $0.0000000083 x86_64)
// = 19.3% cheaper for same duration
// IMPORTANT: container images must be ARM64 architecture
// Build: docker buildx build --platform linux/arm64 -t shopflow-fn:arm64 .
// Verify: docker inspect shopflow-fn:arm64 | grep Architecture → arm64
// For Java 21: no code changes needed — JVM JIT generates ARM instructions
// For native image: recompile with -H:Platform=linux-aarch64
```

Optimization	Mechanism	ShopFlow Impact	Effort
ARM_64 (Graviton2)	20% cheaper per GB-ms	\$0.83 vs \$1.00 per M GB-ms	CDK one-liner + ARM container rebuild
SnapStart	Eliminate 3-8s cold start	API Gateway p99 <200ms	Requires published version + alias
Lambda Power Tuning	Find optimal memory for cost/speed	20-40% cost reduction per function	AWS Step Functions automation (1-t)
Minimize package size	Smaller ZIP = faster download, less cold start	<1s	Maven dependency audit + layers
X-Ray sampling	Reduce X-Ray trace cost	X-Ray at \$5/M traces; 5% sampling = \$0.25/M	Tracing config change

■ AWS Lambda Power Tuning is a free open-source tool (AWS Serverless Application Repository) that uses Step Functions to run your Lambda at multiple memory settings with real payloads and measures duration + cost. Deploy via SAR (sar: powertuning), run with your function ARN, receive a visualization of cost vs performance. Run before committing to a memory setting in production.

09-11

Lambda X-Ray Tracing & Structured Logging

AWS X-Ray traces end-to-end request flow across API Gateway → Lambda → Aurora/Redis/DynamoDB. Lambda Powertools provides structured logging (JSON to CloudWatch Logs), custom metrics via EMF (Embedded Metric Format), and X-Ray subsegment tracing with zero-code instrumentation.

Lambda Powertools for Java — Tracing, Logging, Metrics

```
// Powertools annotations: add @Tracing, @Logging, @Metrics
@Logging(logEvent=true)
@Tracing
@Metrics(namespace="ShopFlowLambda",service="price-calculator")
public class PriceHandler
implements RequestHandler<APIGatewayProxyRequestEvent,APIGatewayProxyResponseEvent> {
  private static final Logger log = LogManager.getLogger(PriceHandler.class);
  private static final Metrics metrics = MetricsFactory.getMetricsInstance();
  @Override
  public APIGatewayProxyResponseEvent handleRequest(
    APIGatewayProxyRequestEvent event, Context context) {
```

```
// Structured log – JSON in CloudWatch Logs
log.info("Processing price request",
Map.of("productId", event.getPathParameters().get("productId"),
"userId", context.getIdentity().getIdentityId(),
"requestId", context.getAwsRequestId()));
// Custom metric – count of personalized price calculations
metrics.addMetric("PersonalizedPriceCalculation",
MetricUnit.COUNT, 1);
// X-Ray subsegment for Aurora query
Subsegment segment = AWSXRay.beginSubsegment("Aurora::getBasePrice");
try {
PriceDto price = priceService.getBasePrice(event);
return buildResponse(200, price);
} finally {
AWSXRay.endSubsegment();
}
}
}
```

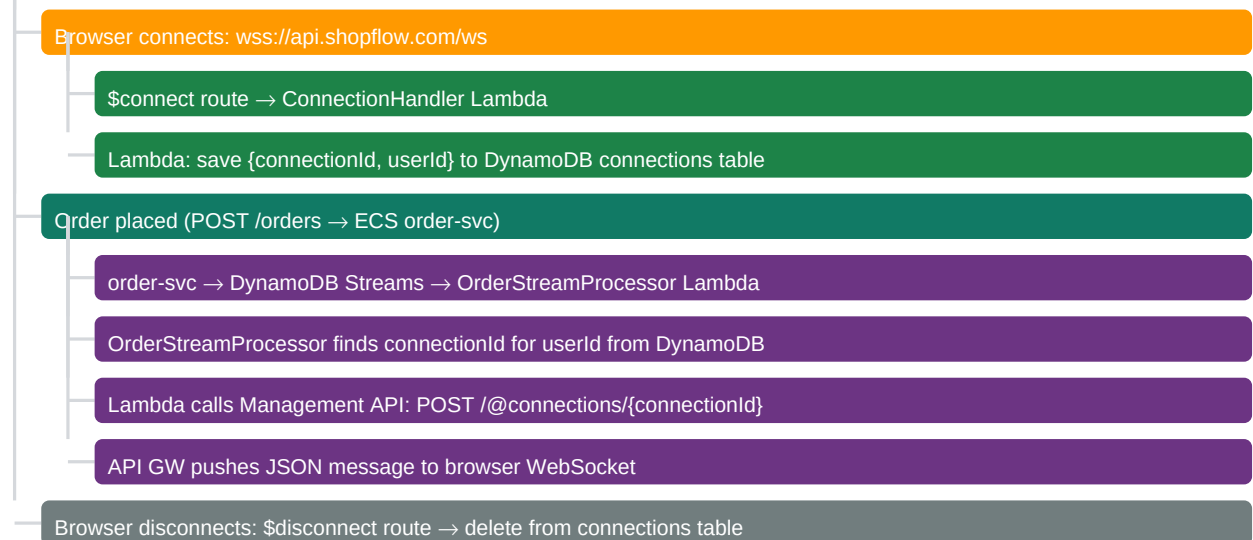
■ Lambda Powertools structured logging automatically adds `traceId`, `requestId`, `functionName`, `functionVersion` to every log line. This enables CloudWatch Logs Insights to correlate all log lines for one request: fields `@requestId`, `@traceId` | filter `@requestId = "abc-123" | stats count(*)`. Without structured logging, finding all log lines for one API request requires full-text search across all log groups.

09-12

WebSocket API — Real-Time Order Status

ShopFlow uses API Gateway WebSocket API for real-time order status updates pushed to the browser during checkout. Lambda manages WebSocket connections; when order status changes (PENDING → CONFIRMED → SHIPPED), Lambda pushes updates to the connected client via Management API.

WebSocket API — Order Status Flow



WebSocket Push — Management API

```
// OrderStreamProcessor: push status update to browser
String endpoint = "https://" + apiId + ".execute-api.us-east-1.amazonaws.com/prod";
ApiGatewayManagementApiClient mgmtApi = ApiGatewayManagementApiClient.builder()
.region(Region.US_EAST_1)
.endpointOverride(URI.create(endpoint))
.build();
String messageJson = json.writeValueAsString(Map.of(
```

```
"event", "ORDER_STATUS_UPDATE",
"orderId", orderId,
"status", "CONFIRMED",
"eta", "2024-12-01"
mgmtApi.postToConnection(PostToConnectionRequest.builder()
.connectionId(connectionId)
.data(SdkBytes.fromUtf8String(messageJson))
.build());
```

WebSocket Route	Handler Lambda	Action	DynamoDB Table
\$connect	ConnectionHandler	Save connectionId + userId + TTL=2hr	shopflow-ws-connections
\$disconnect	ConnectionHandler	Delete connectionId from table	shopflow-ws-connections
order-status (custom)	OrderStatusRouter	Push status update to specific connectionId	Read connectionId by userId
ping (heartbeat)	ConnectionHandler	Extend TTL in DynamoDB (prevent idle disconnect)	shopflow-ws-connections

■ API Gateway WebSocket connections time out after 10 minutes of idle (no messages either direction). Implement a client-side heartbeat: browser sends ping every 5 minutes, server responds and updates DynamoDB TTL. Without heartbeat, users with the checkout page open for >10 minutes will silently lose the connection and miss order status updates.

09-13

Lambda EventBridge Scheduler — Batch Jobs

EventBridge Scheduler (2022) replaces CloudWatch Events for Lambda scheduling. It supports cron expressions, rate expressions, and one-time schedules. ShopFlow uses it for daily sales reports, hourly cache warming, and weekly catalog export — replacing legacy cron-on-EC2 patterns.

Job	Schedule	Lambda	Memory / Timeout
Daily sales report	cron(0 6 * * ? *) — 6am UTC daily	DailyReport	512MB / 15min
Hourly cache warm (top 100 products)	rate(1 hrs)	CacheWarmer	256MB / 5min
Weekly catalog export to S3	cron(0 2 ? * MON *) — 2am Monday	CatalogExporter	1024MB / 15min
Black Friday pre-warm (one-time)	cron(2024-11-28T07:00:00) — one-time	LambdaPreWarmer	128MB / 2min

EventBridge Scheduler CDK

```
const dailyReportSchedule = new scheduler.Schedule(this,"DailyReport",{
  scheduleName: "shopflow-daily-sales-report",
  schedule: scheduler.ScheduleExpression.cron({
    hour: "6", minute: "0", timeZone: TimeZone.of("UTC"),
  }),
  target: new targets.LambdaInvoke(dailyReportFn,{
    input: scheduler.ScheduleTargetInput.fromObject({
      reportType: "DAILY_SALES",
      date: "<aws.scheduler.scheduled-time>", // injected by Scheduler
    }),
    retryPolicy: {
      maximumRetryAttempts: 3,
      maximumEventAgeInSeconds: 3600, // discard event if 1hr old
    },
  }),
  flexibleTimeWindow: {
    // Allow ±15min execution window – reduce Lambda cold start clustering
    mode: scheduler.FlexibleTimeWindowMode.FLEXIBLE,
    maximumWindowInMinutes: 15,
  },
});
```

■ Use flexibleTimeWindow when exact execution time is not required. EventBridge Scheduler distributes executions across the window — prevents all daily jobs from cold-starting simultaneously. For a Black Friday pre-warm one-time schedule: use at(datetime) expression with flexibleTimeWindow.OFF (exact time required). One-time schedules auto-delete after execution.

09-14

Lambda IAM Permissions — Least Privilege

Each Lambda function has a unique IAM execution role with only the permissions required. ShopFlow uses CDK's built-in grant* methods for common permissions and manual PolicyStatements for fine-grained access. Resource-based policies on Lambda restrict which services can invoke each function.

PriceCalculator IAM Role — CDK

```
// CDK: dedicated execution role (automatically created by Function construct)
// Grant only what this function needs:
// 1. Read from ElastiCache (via VPC — no IAM required, AUTH token in Secrets Manager)
redisAuthSecret.grantRead(priceCalcFn); // read AUTH token
// 2. Read from DynamoDB product cache
productCacheTable.grantReadData(priceCalcFn); // GetItem only
// 3. Read from Aurora (via RDS Proxy IAM Auth)
priceCalcFn.addToRolePolicy(new iam.PolicyStatement({
  actions: ["rds-db:connect"],
  resources: [
    `arn:aws:rds-db:us-east-1:${account}:dbuser:${rdsProxyId}/price_readonly_user`,
  ],
}));
// 4. Write X-Ray traces
priceCalcFn.addToRolePolicy(new iam.PolicyStatement({
  actions: ["xray:PutTraceSegments", "xray:PutTelemetryRecords"],
  resources: ["*"],
}));
// 5. Write CloudWatch Logs (auto-granted by Lambda execution role)
// priceCalcFn.role already has logs:CreateLogGroup + logs:PutLogEvents
```

Lambda Function	Resources It Accesses	IAM Permissions Required
PriceCalculator	DynamoDB product-cache (read), Redis (via VPC), Secrets Manager	GetSecretValue on redis-auth-token; rds-db:connect
ImageResizer	S3 source bucket (read), S3 processed bucket (write)	GetObject (write)/*; PutObject on processed/*; no DB access needed
SearchIndexer	Aurora (read/write), OpenSearch (write)	rds-db:connect; es:ESHttpPost to OpenSearch domain
OrderStreamProcessor	DynamoDB connections table, API Gateway, SQS	execute-api:ManageConnections; SQS:SendMessage

■ Lambda functions in a VPC can access AWS services via VPC endpoints (S3, DynamoDB, Secrets Manager, Kinesis) without traversing the internet or NAT Gateway. Enable VPC endpoints in your CDK stack for each service your Lambda accesses. Lambda in VPC + S3 Gateway endpoint: no \$0.045/GB NAT charge for S3 object access — free within the same region via the endpoint.

09-15

Lambda Monitoring — Metrics, Alarms & Cost

Key Lambda metrics: Duration, Errors, Throttles, ConcurrentExecutions, and InitDuration (cold starts). ShopFlow alarms on error rate above 1%, throttles above 0, and p99 Duration above 3 seconds. Lambda cost: \$0.20 per million invocations + \$0.0000000083 per GB-ms.

Metric	Namespace	Alert Threshold	Root Cause
Errors	AWS/Lambda	Error rate > 1% for 5 min	Handler exception; DB timeout; permission denied
Throttles	AWS/Lambda	Any non-zero	Concurrency limit hit; increase reserved concurrency
Duration p99	AWS/Lambda	> 3s for API GW functions	Cold start; slow DB query; upstream timeout

Metric	Namespace	Alert Threshold	Root Cause
InitDuration	AWS/Lambda	> 500ms (SnapStart target)	SnapStart misconfigured; large static init
ConcurrentExecutions	AWS/Lambda	> 80% of reserved	Traffic spike; SQS backlog growing; scale up reserved
DeadLetterErrors	AWS/Lambda	Any non-zero	DLQ permission denied; SQS full — check DLQ retention

Monthly Cost Calculation

```
// PriceCalculator: 5M invocations/month, avg 150ms @ 256MB
// Invocation cost: 5M × $0.20/M = $1.00
// Duration cost: 5M × (256/1024) × 0.150s × $0.0000000083 = $0.00016/M
// = 5M × $0.00016/M = $0.80
// Subtotal PriceCalculator: $1.80/month
// ImageResizer: 200K invocations, avg 5000ms @ 1024MB
// Invocation: 200K × $0.20/M = $0.04
// Duration: 200K × (1024/1024) × 5.0s × $0.0000000083 = $0.0083/M = $0.83
// Subtotal ImageResizer: $0.87/month
// Free tier: 1M invocations + 400,000 GB-seconds/month
// ShopFlow all Lambdas total: ~$8/month (well within control plane budget)
// vs equivalent EC2/ECS: $50-200/month for async workers
```

Lambda vs ECS	Lambda	ECS Fargate
Billing model	Per invocation + GB-ms (pay exactly for what used)	Per CPU/memory hour (pay when idle too)
Scaling speed	0 → 1,000 concurrent in seconds	Minutes (ECS service scale-out)
Max execution duration	15 minutes	Unlimited (long-running)
Cold start	0ms (warm) to 500ms (SnapStart)	ECS pull + task start: 30-120s for new task
Connection reuse	Per execution environment (not per invocation)	Per container (reliable persistent connections)
Best for ShopFlow	Async event processing, short stateless functions	Synchronous APIs, long-running services, stateful

■ Lambda is most cost-effective for workloads with intermittent bursts. DailyReport Lambda at 30 invocations/month: \$0.000006 vs equivalent EC2 t3.medium cron: \$30/month. The break-even is approximately 500,000 invocations per month per 512MB function for equivalent EC2. Below that, Lambda is cheaper; above that, ECS Fargate becomes more cost-effective for sustained workloads.

09-BP

Best Practices & Anti-Patterns

- ✓ Use SnapStart for all Java Lambda functions behind API Gateway — eliminates 95%+ of cold start latency
- ✓ Initialize DB/cache connections in static {} blocks — reused across invocations, not re-created per invocation
- ✓ Use ARM_64 (Graviton2) for all Lambda functions — 20% cheaper, same performance, no code changes for Java
- ✓ Configure DLQ for all async Lambda triggers (S3, EventBridge, SNS) — no silent event drops
- ✓ Use bisectOnError=true for DynamoDB Streams and Kinesis — isolate poison-pill records via batch splitting
- ✓ Use HTTP API (v2) instead of REST API unless specifically needing request validation or WAF direct attach
- ✓ Implement circuit breaker in Lambda for Redis and Aurora calls — cache unavailability should not cause function failure
- ✓ Use RDS Proxy for all Lambda-to-Aurora connections — prevent connection exhaustion at high concurrency
- ✓ Log structured JSON (Lambda Powertools) — enables CloudWatch Logs Insights correlation by requestId
- ✓ Use EventBridge Scheduler with flexibleTimeWindow for non-time-sensitive batch jobs — reduce cold start clustering
- ✓ Assign least-privilege IAM role per function — never share roles across multiple functions
- ✓ Use Lambda Layers for shared dependencies — reduce package size, improve cold start time
- ✓ Set reserved concurrency for critical functions — isolate from noisy-neighbor Lambda bursts
- ✓ Enable X-Ray tracing with 5% sampling in production — trace distributed calls without full cost

- ✓ Monitor InitDuration metric — alert when >500ms for SnapStart functions; indicates misconfiguration
- ✓ Size memory using Lambda Power Tuning tool — find cost/speed optimum for each function
- Never use Lambda with direct JDBC to Aurora without RDS Proxy — connection exhaustion at scale is guaranteed
- Do not put long-running business logic (>15min) in Lambda — use ECS Fargate, Step Functions, or Batch
- Avoid sharing IAM execution roles across functions — least-privilege requires one role per function
- Do not call AWS SDK synchronously inside Lambda without timeout — default SDK timeout can exceed Lambda timeout
- Never hardcode credentials in Lambda code or environment variables — use IAM roles and Secrets Manager
- Do not use Lambda for continuous polling — use SQS trigger or EventBridge; polling wastes invocation budget

09-QR

Quick Reference & Interview Q&A

Concept	Key Value / Rule
SnapStart cold start	3-8s Java cold start → 50-200ms with SnapStart. Requires published version + alias. Re-run network in
Lambda timeout	15 minutes maximum. API Gateway integration timeout: 29 seconds (hard limit regardless of Lambda ti
Lambda concurrency limits	Account: 1,000 default (soft limit). Burst: +500/min to 3,000 (us-east-1). Reserved: guarantee + cap per
HTTP API vs REST API	HTTP API: 70% cheaper, lower latency, JWT/Lambda/CORS built-in. REST API: usage plans, WAF dir
Lambda DLQ types	Async triggers (S3, EventBridge, SNS): Lambda DLQ. Event source mapping (SQS, Streams): configur
VPC Lambda cold start	Lambda in VPC adds 200-500ms to first cold start (ENI creation). With Hyperplane ENI (2019+), subse
ARM_64 pricing	\$0.0000000067 per GB-ms (vs \$0.0000000083 x86). 19.3% cheaper. Same billing model. Requires AR
API Gateway throttling 429	Rate limit: sustained RPS. Burst limit: tokens for spike absorption. Client receives 429 Too Many Reque
WebSocket connection TTL	API GW WebSocket idle timeout: 10 minutes. Maximum duration: 2 hours. Send heartbeat (ping) every

Top Interview Questions

Question	Answer
How does SnapStart work?	SnapStart: Lambda pre-runs the INIT phase (JVM start + static init + class loading) once after publish. Ser
Lambda vs ECS Fargate — when to use?	Lambda: Stateless functions, event-driven, <15min, variable/bursty traffic, pay-per-use. ECS Fargate: lon
How to fix Lambda throttling?	First: identify which function is throttling (CloudWatch Metrics by FunctionName). Then: (1) Increase reser
RDS Proxy + Lambda — why required?	1 connection per execution environment × 1,000 concurrent = 1,000 connections → exceeds Aur
API GW 29-second timeout — why?	API Gateway has a hard limit of 29 seconds. For functions >29s: (1) Async pattern: API GW returns 202 Accep